

```
--- djangoapi\scripts\crop\parcelas\parcelasP00.py ---
```

```
from psycopg.rows import dict_row
from myLib.gestor import tablasP00
```

```
class ParcelasP00(tablasP00):
```

```
    # INSERT
```

```
    def insert(self, d: dict):
        wkt = d['geom_wkt']
```

```
        if not self.geom_isValid(wkt):
            print("Error: La geometría no es válida (self-intersection, etc.)")
            self.disconnect()
            return None
```

```
        if self.has_interior_intersection('parcelas', wkt):
            print("Error: La parcela intersecta el interior de una parcela existente.")
            self.disconnect()
            return None
```

```
        geom_final    = self.snap_to_grid(wkt)
        dueno          = d['dueno']
        area_m2        = d['area_m2']
        cultivo         = d['cultivo']
        fecha_siembra  = d['fecha_siembra']
```

```
        cons = """
            INSERT INTO parcelas
                (dueno, area_m2, cultivo, fecha_siembra, geom)
            VALUES
                (%s, %s, %s, %s, ST_GeomFromText(%s, %s))
            RETURNING id
        """
```

```
        try:
            self.cur.execute(cons, [dueno, area_m2, cultivo, fecha_siembra,
                                    geom_final, self.epsg])

            self.conn.commit()
            new_id = self.cur.fetchone()[0]
            print(f"Parcela de '{dueno}' insertada con id: {new_id}")
            return new_id
```

```
        except Exception as e:
            print(f"Error en inserción de parcela: {e}")
            self.conn.rollback()
            return None
```

```
        finally:
            self.disconnect()
```

```
    # SELECTALL
```

```
    def selectAll(self, d: dict):
```

```

id_min = d['id_min']

cons = """
    SELECT COUNT(*) AS total
    FROM parcelas
    WHERE id > %s
"""
try:
    self.cur.execute(cons, [id_min])
    total = self.cur.fetchone()[0]
    print(f"{total} parcela(s) encontrada(s).")
    return total

except Exception as e:
    print(f"Error en selectAll de parcelas: {e}")
    return 0

finally:
    self.disconnect()

# SELECTASTUPLE
def selectAsTuple(self, d: dict):
    id_min = d['id_min']

    cons = """
        SELECT id, dueno, area_m2, cultivo, fecha_siembra,
               ST_AsText(geom) AS geom_wkt
        FROM parcelas
        WHERE id > %s
    """
    try:
        self.cur.execute(cons, [id_min])
        rows = self.cur.fetchall()
        print(f"{len(rows)} parcela(s) encontrada(s).")
        return rows

    except Exception as e:
        print(f"Error en selectAsTuple de parcelas: {e}")
        return []

    finally:
        self.disconnect()

# SELECTASDICT
def selectAsDict(self, d: dict):
    id_min = d['id_min']

    cons = """
        SELECT id, dueno, area_m2, cultivo, fecha_siembra,
               ST_AsText(geom) AS geom_wkt
        FROM parcelas
        WHERE id > %s
    """

```

```

try:
    self.cur = self.conn.cursor(row_factory=dict_row)
    self.cur.execute(cons, [id_min])
    rows = self.cur.fetchall()
    print(f"{len(rows)} parcela(s) encontrada(s).")
    return rows

except Exception as e:
    print(f"Error en selectAsDict de parcelas: {e}")
    return []

finally:
    self.disconnect()

```

# UPDATE

```

def update(self, d: dict):
    wkt = d['geom_wkt']

    if not self.exists_by_id('parcelas', d['id']):
        print(f"Error: No existe la parcela con id {d['id']}.")
        self.disconnect()
        return None

    if not self.geom_isValid(wkt):
        print("Error: La geometria no es valida.")
        self.disconnect()
        return None

    geom_final = self.snap_to_grid(wkt)

    if self.has_interior_intersection_excluding_id('parcelas', geom_final, d['id']):
        print("Error: La parcela intersecta el interior de una parcela existente.")
        self.disconnect()
        return None

    cons = """
        UPDATE parcelas
        SET
            dueno          = %s,
            area_m2        = %s,
            cultivo        = %s,
            fecha_siembra  = %s,
            geom            = ST_GeomFromText(%s, %s)
        WHERE id = %s
    """
    try:
        self.cur.execute(cons, [
            d['dueno'], d['area_m2'], d['cultivo'], d['fecha_siembra'],
            geom_final, self.epsg, d['id']
        ])
        self.conn.commit()
        print(f"Parcela actualizada: {self.cur.rowcount} fila(s).")
    
```

```
except Exception as e:
    print(f"Error en update de parcelas: {e}")
    self.conn.rollback()

finally:
    self.disconnect()

# DELETE
def delete(self, d: dict):
    cons = "DELETE FROM parcelas WHERE id = %s"
    try:
        self.cur.execute(cons, [d['id']])
        self.conn.commit()
        print(f"Parcelas eliminadas: {self.cur.rowcount} fila(s).")

    except Exception as e:
        print(f"Error en delete de parcelas: {e}")
        self.conn.rollback()

    finally:
        self.disconnect()
```